

Relatório final — Configuração do ambiente e fluxo de trabalho com IA

Aluno: Lucas Santos **Disciplina:** Tópicos Avançados em Engenharia de Software 2 — PPgTI / IMD-UFRN **Repositório:** <https://github.com/lucassmsantoss/ia-dev-lab>
Pull Request: <https://github.com/lucassmsantoss/ia-dev-lab/pull/1> (aberto, sem merge) **Data:** 27/08/2026

1. Ferramentas configuradas e por quê

Configurei duas ferramentas, uma de cada categoria pedida. **VS Code + GitHub Copilot** para a categoria IDE + assistente, por já estar instalado e autenticado na máquina e por ser o formato certo para o ciclo curto de escrita, em que o assistente enxerga o cursor e o arquivo aberto. **Claude (Cowork)** para a categoria CLI agent, pelo ciclo longo: criar estrutura de pastas, escrever documentação consistente com o código e alterar vários arquivos de uma vez.

O que decidi a escolha do Claude não foi a geração de código em si, e sim o arquivo de contexto de projeto. Poder declarar as convenções uma única vez, versionadas no repositório, em vez de repeti-las a cada prompt, é o que torna o experimento da Etapa 4 mensurável. O raciocínio completo, com as alternativas descartadas (Cursor, Codex CLI, Continue) e as consequências negativas assumidas, está no [ADR 0001](#).

2. Trecho mais útil do CLAUDE.md

O trecho de maior efeito prático não está no `CLAUDE.md` da raiz, e sim na regra de escopo em `src/validacao/CLAUDE.md`:

A função **normaliza a entrada antes de validar**: remove pontos, traços, barras e espaços. Entrada inválida — `None`, string vazia, tamanho errado, caractere não numérico — retorna `False`. Nunca levanta exceção. Sequências de dígitos repetidos (`"1111111111"`) são rejeitadas, mesmo quando passam no cálculo do dígito verificador.

(Citação da versão vigente em 27/08/2026. O arquivo foi ampliado depois, na atividade de Spec-Driven Development, para cobrir também o CNPJ alfanumérico.)

Ele é o mais útil porque define o **contrato** da função, não o seu estilo. Regras de estilo (nomes, tipagem, tamanho de linha) tornam o código mais agradável; essas três linhas evitam três bugs concretos, sendo um deles um falso positivo de validação — o pior defeito possível em um validador. É também a parte que, uma vez escrita, deixa de precisar ser digitada em todo prompt.

3. Diferença entre o prompt fraco e o prompt eficaz

A diferença entre os dois prompts não foi a quantidade de código pedida, e sim o quanto ficou declarado — no prompt eficaz eu entreguei o porquê da tarefa, o contexto do projeto e as regras que a validação precisa respeitar. Rodei as duas versões geradas contra os mesmos dez casos: o prompt fraco acertou **4 de 10**, o eficaz acertou **10 de 10**, embora o cálculo dos dígitos verificadores estivesse correto nas duas versões. O modelo já tinha o conhecimento do domínio; faltava o contrato.

As seis falhas do prompt fraco foram todas suposições que o modelo teve que fazer sozinho porque nada as fixou: supôs que a entrada viria sem máscara (falso negativo em CPF com pontuação), que `"1111111111"` era válido (falso positivo, o pior defeito possível em um validador) e que lançar exceção em entrada nula era aceitável. Ou seja, o prompt eficaz não fez o modelo interpretar mais — fez ele interpretar **menos**, porque cada restrição substituiu uma adivinhação por uma regra.

Foi por isso que a **restrição** rendeu mais que os outros elementos: cada uma das quatro eliminou exatamente uma classe de falha. O exemplo de padrão mudou a forma do código (assinatura, tipagem, função auxiliar isolada), o que ajuda na manutenção, mas sozinho não teria corrigido nenhum dos seis casos. E a **forma de validar** mudou a natureza da conclusão: ao pedir os testes junto com a implementação, a qualidade deixou de ser opinião e virou número verificável.

Ao repetir o prompt fraco em outra ferramenta (GitHub Copilot), já com o repositório contendo o `CLAUDE.md`, apareceu o resultado mais interessante do trabalho: o agente **não** gerou código ingênuo — leu a implementação existente, rodou os testes e reportou `38 passed`. O mesmo prompt vago que antes acertava 4 de 10 passou a se comportar corretamente, porque as decisões que antes ele precisava adivinhar já estavam escritas no repositório. **Contexto versionado eleva o piso do prompt fraco**: um `CLAUDE.md` bem feito funciona como um prompt permanente, que paga o custo de escrita uma vez e desconta em todos os pedidos seguintes. A comparação não é controlada — mudaram a ferramenta e a presença do contexto ao mesmo tempo — e essa limitação está registrada em `prompts-comparacao.md`, junto com o detalhamento completo do experimento.

4. Obstáculo enfrentado e como resolvi

Nenhum obstáculo veio da IA. Todos vieram do **ambiente Windows**, em cinco erros encadeados: o alias da Microsoft Store interceptando o comando `python`; `ImportError: DLL load failed while importing ssl` ao contornar isso com o caminho absoluto do Anaconda; a política de execução do PowerShell bloqueando o `npx.ps1`; o `pip` invisível fora do Anaconda Prompt; e o `mcp-server-git` recusando instalação.

Os quatro primeiros são o mesmo problema: a ferramenta estava instalada, mas o terminal não sabia disso. Resolvi de vez com `conda init powershell`, que passou a ativar o ambiente automaticamente, mais `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` para liberar scripts locais. O quinto era de outra natureza — o `mcp-server-git` exige Python 3.10+ e o ambiente base é 3.9.12 — e optei por documentar a tentativa em vez de reconfigurar o ambiente a essa altura, mantendo o servidor de filesystem, que atende ao requisito e funciona.

O que aprendi aqui não é sobre PATH. Nos cinco casos, a mensagem de erro apontava para o sintoma e não para a causa: “python não foi encontrado” com o Python instalado; “DLL não encontrada” quando o problema era ativação de ambiente; `from versions: none` quando o pacote existe e o incompatível era o interpretador. A IA acelerou muito a tradução de sintoma para causa. Mas a decisão sobre **qual** caminho seguir em cada bifurcação — instalar mais uma ferramenta, criar um ambiente novo, ou aceitar um servidor a menos e documentar — continuou sendo minha, porque dependia de restrições que não estão no código: o prazo e o quanto valia sujar o ambiente da máquina.

Checklist de entrega

- ✓ Repositório no GitHub com histórico de commits
- ✓ Arquivo `CLAUDE.md` e ao menos uma regra customizada (`src/validacao/CLAUDE.md`)
- ✓ Estrutura de pastas organizada, `README.md` e um ADR em `docs/adr/`
- ✓ Arquivo `docs/prompts-comparacao.md` com o comparativo de prompts
- ✓ Pull Request aberto no GitHub (#1, sem merge)
- ✓ Arquivo `.mcp.json` — servidor de filesystem configurado e testado
- ✓ Relatório final em Markdown e PDF

Evidência de execução: `python -m pytest tests/test_cpf.py -q` → `38 passed` (Anaconda, Python 3.9.12, pytest 7.1.1).